

VISUAL EXPLANATION ENGINE

시스러운 카드 설명을 줄이기 위한 연구 보고서

여러 설명 fixture를 만들고, 브라우저 캡처와 heuristic metric으로 비교하면서 skill 지침을 반복 개선했다. 핵심 목표는 "박스 안에 박스"와 detached prose card를 줄이고, 주제의 native surface가 이해를 직접 운반하게 만드는 것이다.

10

benchmark pairs

31 -> 79

average overall

+48

average lift

Run: 2026-06-28T14-54-10-487Z · Generated: 2026. 6. 28. 오후 11:54:29

RESEARCH LOOP

Generate examples, capture screenshots, score regressions, inspect visually, then tighten the skill.

box

container pressure

visual

surface dominance

labels

collision checks

state

interaction change

MIDDLE PROCESS

개선은 한 번의 디자인이 아니라 압력 루프였다

8d64f81

Research harness

스크린샷, DOM metric, interaction diff를 같은 run report로 수집.

7543b86

Anti-box pressure

box area, nested cards, rounded labels, collision metric으로 AI 스러운 카드 UI를 압박.

1c201e6

Code trace benchmark

React render 설명을 lifecycle 카드가 아니라 code/runtime/output trace로 전환.

bfb395

Science mechanism benchmark

정의 카드 대신 chloroplast cross-section과 energy/matter flow를 검증.

362a8a6

Timeline causality

incident postmortem을 event cards가 아닌 scaled timeline으로 비교.

9427d0b

Architecture tradeoff

pros/cons 카드 대신 constraint decision surface를 추가.

962c26a

Data analytics

KPI dashboard 대신 cohort heatmap과 retention curve lens를 추가.

1. Primary object first

그래프, 코드, 타임라인, cohort grid, decision surface가 화면을 이끌고, 텍스트는 annotation으로 붙는다.

2. Cards are suspect

카드는 진짜 domain object일 때만 쓴다. 설명 카드가 많아질수록 box discipline이 떨어진다.

3. Interaction must change the object

버튼은 paragraph를 바꾸는 장식이 아니라 selected state, path, segment, point, surface를 바꿔야 한다.

4. Labels must not cover evidence

하네스는 label collision을 잡고, 시각 QA는 실제 screenshot에서 겹침을 다시 확인한다.

5. Native surface beats metaphor

알고리즘은 그래프, 평가는 산출물, 과학은 구조, 데이터는 데이터 표면으로 설명한다.

6. Metrics guide, eyes decide

점수는 압력 장치이고, 최종 판단은 캡처를 보며 과밀함과 이해 가능성을 확인한다.

QUANTITATIVE SUMMARY

카드형 baseline 대비 visual-first candidate가 안정적으로 상승

TASK	BASELINE	CANDIDATE	DELTA
Dijkstra Lab Algorithm	35	76	+41
Rubric Inspection Evaluation Language	24	71	+47
OAuth Request Path System/API Flow	34	76	+42
Gradient Descent Surface Math	35	85	+50
React Render Debugging Code Debugging	30	83	+53
Photosynthesis Mechanism Science	32	79	+47
Incident Timeline Timeline/Causality	27	78	+51
Architecture Tradeoff Architecture	26	77	+51
Retention Analysis Data/Analytics	34	77	+43
Dependency Graph Library-backed Network	32	90	+58

Dijkstra Lab

+41
overall

Baseline

35

Candidate

76

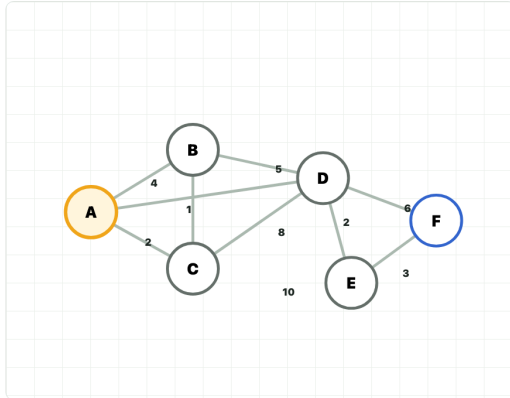
다익스트라 실험실

출발점에서 가장 가까운 후보를 하나씩 확정하며 최단거리를 퍼뜨리는 과정을 눌러서 따라가 보세요.

출발 A ▼ 목표 F ▼ 이전 다음 자동

가중치 그래프

● 현재 선택 ● 거리 확정 ● 목표 경로



확정

A의 거리를 확정합니다

큐에서 가장 작은 값은 A=0입니다. 음수 간선이 없으므로 이 값은 더 줄어 들 수 없습니다.

현재: A = 0

우선순위 큐

큐의 맨 앞 후보가 다음으로 확정될 노드입니다. 값이 작을수록 먼저 나옵니다.

비어 있음

읽는 법

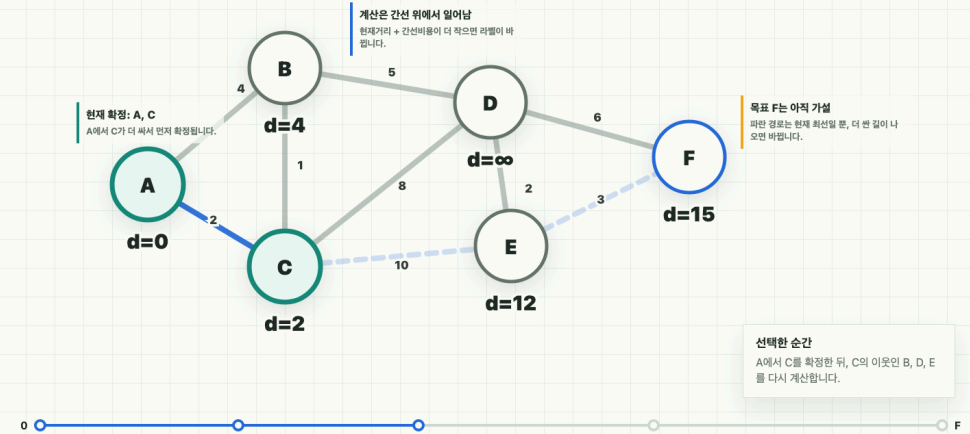
- 1 현재 가장 짧은 후보를 하나 고르면 거리는 확정됩니다.
- 2 그 노드의 이웃을 보며 현재거리 + 비용을 계산합니다.
- 3 세 계산값이 더 작으면 거리와 이전 노드를 바꿉니다.

cards 12 · box 0.90 · labels 0 · lib 0 · hand 1 · libscore 45 · state 0

다익스트라를 그래프 위에서 보기

가장 작은 후보를 확정하면 그 노드에서만 거리가 퍼집니다. 파란 ghost path는 현재까지의 최단 경로 가설입니다.

A에서 시작 ▼ F까지 ▼ 되감기 다음 확정



cards 4 · box 0.04 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

그래프 자체가 화면이 되면 거리 확정, 후보, 경로 변화가 카드 설명 없이 읽힌다.

Open graph-paper stage with attached labels, ghost path, compact floating inspector, and state-changing controls.

Rubric Inspection

+47
overall

Baseline

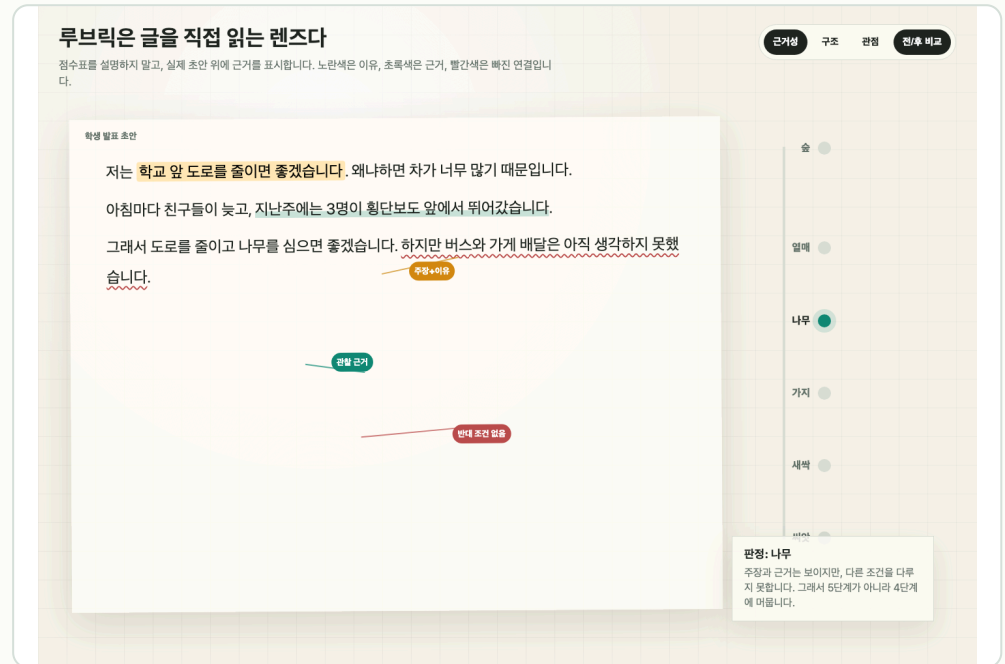


cards 16 · box 0.67 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

24

Candidate

71



cards 5 · box 0.01 · labels 0 · lib 0 · hand 0 · libscore 100 · state 1

What changed

평가 언어는 추상 기준보다 실제 산출물 위 evidence lens로 보여줄 때 강하다.

Artifact-centered rubric lens: annotated excerpt, criterion overlays, evidence trace, and before/after toggle.

OAuth Request Path

+42
overall

Baseline

34

Candidate

76

OAuth 로그인 흐름

사용자, 브라우저, 앱 서버, 인증 제공자가 코드를 주고받아 로그인하는 구조입니다.

등장인물

사용자

로그인 버튼을 누릅니다.

브라우저

리다이렉트를 따라갑니다.

앱 서버

세션을 만듭니다.

Provider

코드와 토큰을 발급합니다.

1

로그인 시작

사용자가 앱에서 로그인 버튼을 누르면 인증 서버로 이동합니다.

2

동의 화면

제공자가 사용자에게 권한 동의를 받습니다.

3

코드 반환

브라우저는 authorization code를 앱 callback으로 가져옵니다.

4

토큰 교환

앱 서버가 코드를 access token으로 바꾸고 세션을 만듭니다.

중요한 전제

authorization code는 access token이 아닙니다. 서버가 provider와 직접 교환해야 합니다.

code ≠ token

보안 포인트

state는 CSRF를 막기 위한 값이고, redirect URI는 미리 등록되어 있어야 합니다.

state 검증

흔한 오해

브라우저가 토큰을 직접 들고 오는 흐름으로 이해하면 서버 세션과 code exchange를 놓칩니다.

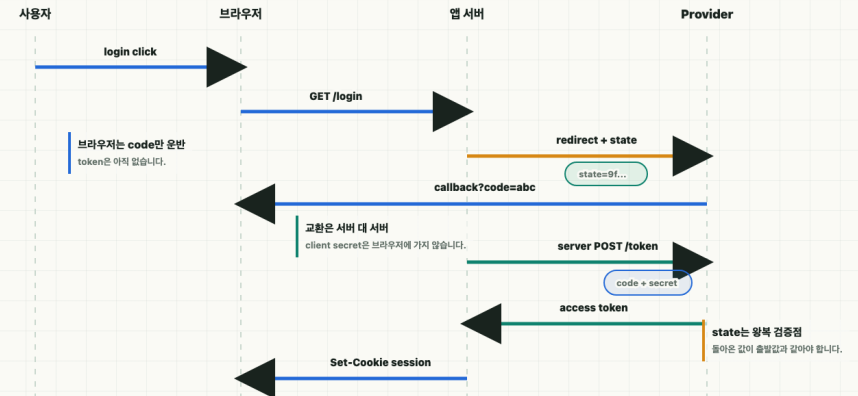
다음 설명

cards 15 · box 1.01 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

OAuth는 카드 목록이 아니라 왕복 경로다

브라우저가 code를 들고 돌아오고, 서버가 provider와 따로 token을 교환합니다. state와 redirect URI는 흐름의 특정 지점에 붙어 있습니다.

처음 token exchange 보기



cards 4 · box 0.03 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

보안 개념은 request path 위의 state, code, token 위치로 붙을 때 이해가 빠르다.

Request trace with tokens and redirects attached directly to the flow.

+50
overall

35

85

현재 위치에서 기울기를 계산하고, 학습률을 곱한 만큼 반대 방향으로 이동합니다. 이 과정을 반복하면 손실이 작아집니다.

함수의 어느 방향이 올라가는지 알려주는 값입니다. 우리는 반대 방향으로 이동합니다.

```
gradient = df/dx
```

한 번에 얼마나 움직일지 정하는 숫자입니다. 너무 크면 지나치고 너무 작으면 느립니다.

```
x_next = x -  $\eta$  · gradient
```

작은 이동을 여러 번 반복하면 최솟값 근처에 도착합니다.

설명 바꾸기

현재 위치

$n = 0.18$ 작게 크게 한 걸음 이동

현재 점의 접선이 올라가는 방향을 말해주면, gradient descent는 그 반대 방향으로 한 걸음 내려갑니다.

이 증가하는 쪽입니다.

다음 x

반대 방향으로 이동
 $x_{next} = x - \eta \cdot \text{gradient}$

목표는 낮은 곳
공식은 곡선 위 이동을 짧게 쓴 것입니다.

최솟값 근처

cards 4 · box 0.03 · labels 0 · lib 0 · hand 0 · libscore 100 · state 1

Curve-first explanation with tangent, step arrow, and visible point update.

React Render Debugging

+53
overall

Baseline

30

Candidate

83

React 렌더링을 이해하기

상태 업데이트, 렌더, 커밋, 이펙트가 어떤 순서로 일어나는지 단계별 카드로 정리했습니다.

렌더링 흐름 핵심 개념

1

이벤트

사용자가 버튼을 누르면 이벤트 핸들러가 실행되고 setState가 예약됩니다.

2

렌더

React는 컴포넌트 함수를 다시 호출해서 다음 UI 모양을 계산합니다.

3

커밋

계산된 차이를 실제 DOM에 반영합니다. 화면 변경은 이 시점에 보입니다.

4

이펙트

화면이 반영된 뒤 useEffect가 실행됩니다. 여기서 외부 동기화를 합니다.

```
function Counter() {
  const [count, setCount] = useState(0);
  useEffect(() => log(count), [count]);
  return <button onClick={() => setCount(count + 1)}>
    {count}
  </button>;
}
```

중요한 점

setState는 값을 바로 바꾸는 명령이 아니라 다음 렌더를 예약하는 신호입니다.

다음 카드

흔한 오해

이벤트 함수 안에서 DOM이 즉시 바뀐다고 생각하면 effect 순서를 잘못 읽게 됩니다.

요약 그림



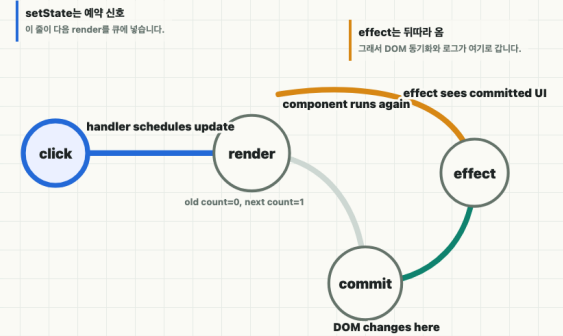
cards 11 · box 1.26 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

React 렌더는 코드가 다시 지나간 흔적이다

버튼 클릭은 값을 즉시 밀어 넣는 일이 아니라, 같은 컴포넌트 함수를 새 상태로 다시 통과시키는 예약입니다.

처음 클릭한 번 실행

```
1 function Counter() {
2   const [count, setCount] = useState(0);
3   useEffect(() => log(count), [count]);
4   return <button
5     onClick={() => setCount(count + 1)}
6     >{count}</button>;
7 }
```



화면은 commit에서 바뀔
비튼 숫자는 여기서 0에서 1로 보입니다.

cards 6 · box 0.06 · labels 0 · lib 0 · hand 0 · libscore 100 · state 1

What changed

코드 라인, runtime state, output, trace를 한 작업면에 연결해야 lifecycle 카드가 필요 없어진다.

Code-and-trace workspace showing setState, render, commit, effect, and visible output change.

Photosynthesis Mechanism

+47
overall

Baseline

32

Candidate

79

광합성은 빛으로 당을 만드는 과정

빛 반응과 캘빈 회로를 단계별 카드로 정리했습니다. 각 단계는 서로 이어져 식물이 에너지를 저장합니다.

광합성 핵심 단계

1

빛 흡수

엽록소가 빛을 받아 전자를 높은 에너지 상태로 올립니다.

2

물 분해

물이 산소, 전자, 수소 이온으로 나뉘고 산소가 밖으로 나갑니다.

3

ATP 생성

전자 흐름과 수소 이온 기울기를 이용해 ATP를 만듭니다.

4

당 합성

CO₂가 캘빈 회로에서 고정되어 당의 재료가 됩니다.



중요한 연결

빛 반응은 ATP와 NADPH를 만들고, 캘빈 회로는 그것을 써서 CO₂를 당으로 바꿉니다.

다음 카드

흔한 오해

식물이 흙을 먹어 몸을 만드는 것이 아니라, 탄소의 대부분은 CO₂에서 옵니다.

요약

빛 에너지 → 화학 에너지 → 탄소 고정이라는 큰 흐름입니다.

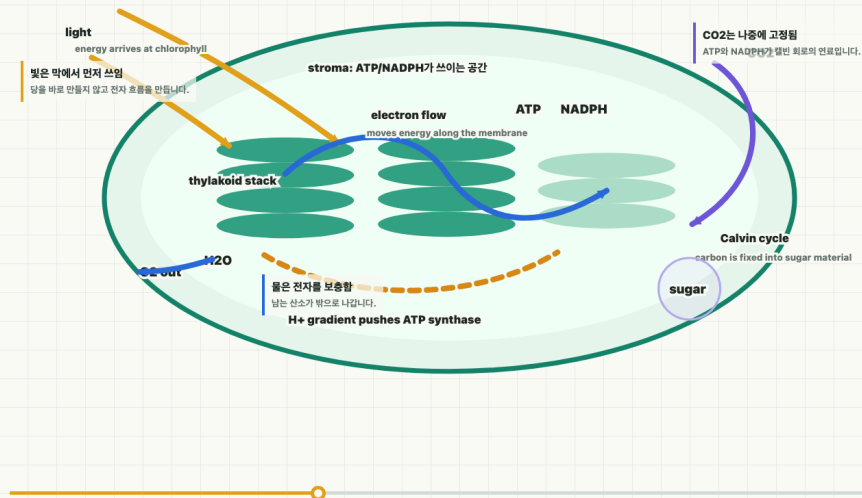
빛 + CO₂ + 물

cards 11 · box 1.12 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

광합성은 엽록체 안의 에너지 물류다

빛은 틸라코이드 막에서 전자 흐름을 만들고, 그 에너지 운반체가 바깥쪽 캘빈 회로에서 CO₂를 당 재료로 묶습니다.

빛 반응 탄소 고정 보기



cards 4 · box 0.03 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

숨은 구조와 matter/energy flow를 직접 붙이면 정의 암기가 아니라 메커니즘으로 읽힌다.

Chloroplast cross-section with light, water, electron, ATP/NADPH, CO₂, and sugar flows attached to structure.

Incident Timeline

+51
overall

Baseline

27

Candidate

78

장애 타임라인 요약

배포 이후 오류율이 올라갔고 롤백 뒤 안정화되었습니다. 중요한 사건을 카드로 정리했습니다.

주요 사건

09:00

배포

새 결제 API 버전이 production에 배포되었습니다.

09:07

DB 대기 증가

커넥션 풀이 포화되면서 요청 대기 시간이 길어졌습니다.

09:14

오류율 급증

사용자는 500 오류를 보기 시작했고 알림이 울렸습니다.

09:32

복구

롤백 이후 지연 시간과 오류율이 정상 범위로 돌아왔습니다.

요약선

원인

새 API가 connection pool을 과도하게 잡아 장애가 발생했습니다.

복구 카드 보기

영향

결제 요청의 일부가 실패했고 support 문의가 증가했습니다.

다음 조치

pool limit guard와 canary 배포를 추가합니다.

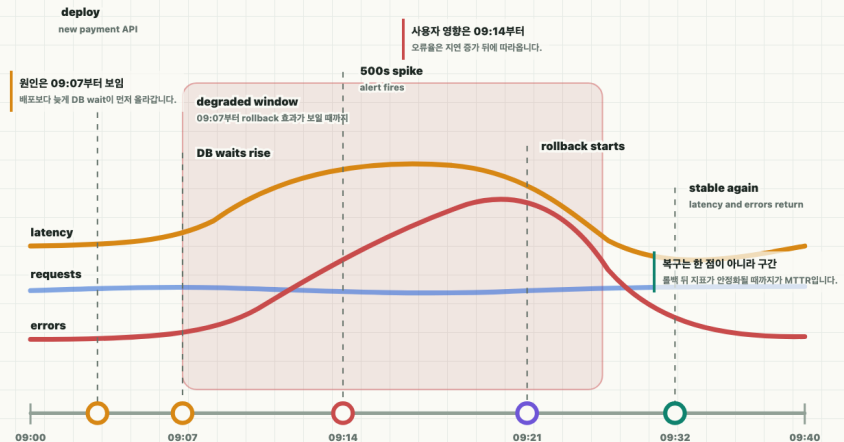
MTTR 32분

cards 16 · box 1.12 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

장애는 카드 목록보다 시간축에서 읽힌다

배포, 지연 증가, 오류 폭증, 롤백, 회복을 같은 축에 놓으면 원인과 MTTR이 한눈에 보입니다.

장애 구간 복구 구간 보기



cards 4 · box 0.04 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

사건 카드를 늘어놓는 대신 time axis, duration, overlap, metric traces가 원인과 MTTR을 보여준다.

Incident timeline with scaled time axis, degraded interval, metric traces, and recovery-state toggle.

Architecture Tradeoff

+51
overall

Baseline

26

Candidate

77

이미지 처리 아키텍처 선택

체크아웃에서 이미지를 처리할 때 단일 API, 서버리스, 큐 워커 중 무엇을 선택할지 장단점을 비교합니다.

후보 옵션

Option A

단일 API 서비스

단순하고 빠르게 만들 수 있습니다.

- 낮은 운영 부담
- 피크 때 API 지연 위험
- 긴 작업에 취약

Option B

서버리스 함수

트래픽 변화에 맞춰 확장됩니다.

- 초기 비용 낮음
- cold start 고려
- 관측성 분산

Option C

큐 워커

작업을 비동기로 안정 처리합니다.

- 버스트 흡수
- 구현 복잡도 증가
- 즉시 응답 아님

중요 기준

지연 시간, 버스트, 비용, 운영 부담을 함께 봐야 합니다.

피크 상황 보기

현재 추천

대부분 상황에서는 큐 워커가 안전해 보입니다.

추천: Queue

주의점

이미지 처리 시간이 길면 사용자 응답과 작업 완료율을 분리해야 합니다.

간단한 포지셔닝

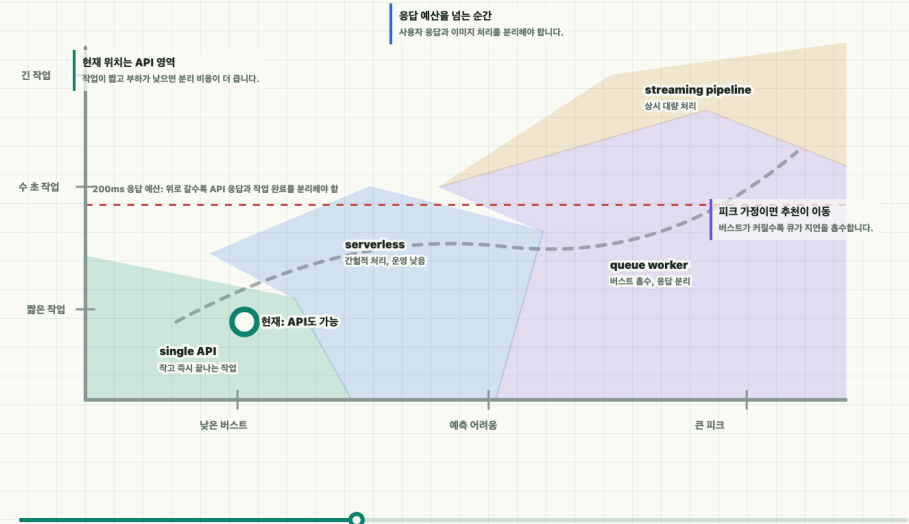


cards 16 · box 1.01 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

아키텍처 선택은 옵션 카드가 아니라 제약의 위치다

체크아웃 이미지 처리는 지연 허용치와 버스트 크기를 같이 보면 추천이 어디서 바뀌는지 드러납니다.

일상 부하 피크 부하로 이동



cards 4 · box 0.04 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

장단점 카드보다 constraint surface 위에서 workload point가 움직일 때 추천 변화가 보인다.

Decision surface with constraint axes, architecture regions, scenario movement, and visual recommendation shift.

Retention Analysis

+43
overall

Baseline

34

Candidate

77

주간 리텐션 하락 분석

새 온보딩 릴리스 이후 리텐션이 낮아졌습니다. 핵심 KPI와 주요 인사이트를 요약했습니다.

요약 지표

Week 1

42%

-9pt

Week 2

31%

-7pt

Activation

58%

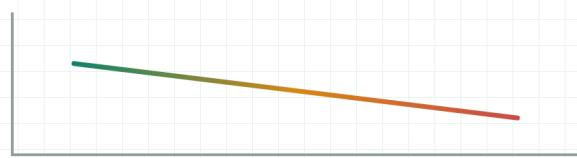
주의

Mobile

39%

하락

간단 추세



핵심 인사이트

온보딩 릴리스 이후 신규 사용자의 유지율이 전반적으로 하락했습니다.

모바일 보기

가능한 원인

새 튜토리얼 단계가 길어져 사용자가 완료 전에 이탈했을 수 있습니다.

추천 조치

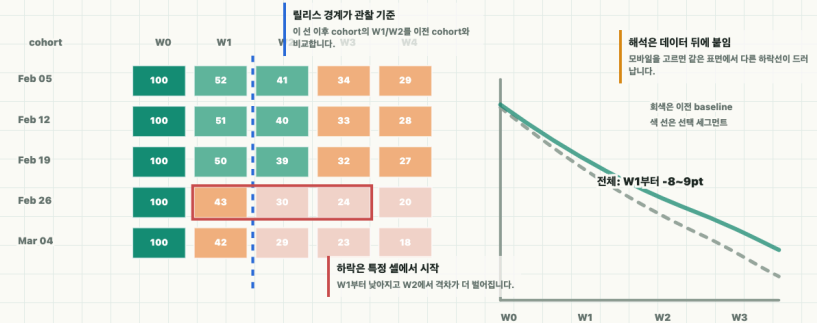
온보딩 단계를 줄이고 모바일 화면의 첫 성공 경험을 앞당깁니다.

cards 14 · box 0.87 · labels 0 · lib 0 · hand 0 · libscore 100 · state 0

리텐션 하락은 KPI가 아니라 cohort 표면에서 보인다

릴리스 전후 cohort를 같은 격자에 놓으면 하락이 전체 문제가 아니라 특정 주차와 세그먼트 문제인지 분리됩니다.

전체 cohort 모바일 세그먼트



cards 4 · box 0.05 · labels 0 · lib 0 · hand 1 · libscore 45 · state 1

What changed

KPI 타일보다 cohort grid, baseline, selected segment, claim annotation 이 같은 데이터 표면에 있어야 한다.

Cohort heatmap and retention curve with release marker, affected cohorts, baseline, and segment toggle.

Dependency Graph

+58
overall

Baseline

32

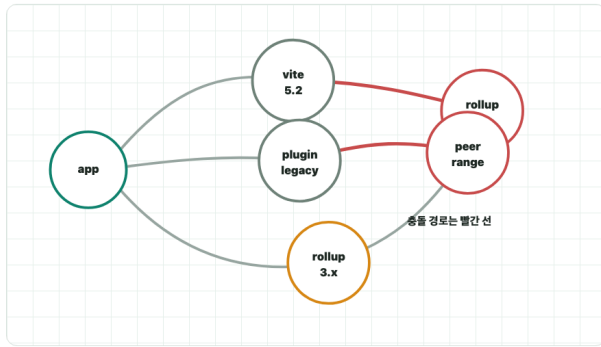
Candidate

90

빌드 의존성 충돌 설명

앱 빌드가 실패한 이유를 의존성 그래프로 요약합니다. 빨간 노드가 충돌 지점입니다.

정적 의존성 그래프



문제

Vite는 Rollup 4를 기대하지만 legacy plugin은 Rollup 3 peer range를 요구합니다.

충돌 카드 보기

왜 실패하나

동일한 빌드 단계에서 두 버전이 섞여 plugin hook shape가 달라집니다.

peer mismatch

조치

legacy plugin을 업데이트하거나 Vite major를 낮춰 peer range를 맞춥니다.

cards 7 · box 0.78 · labels 0 · lib 0 · hand 1 · libscore 45 · state 0

의존성 충돌은 그래프 라이브러리로 탐색해야 한다

Cytoscape가 레이아웃, hit-testing, 선택, 이웃 강조를 맡고 설명은 충돌 노드와 경로에 붙습니다.

전체 그래프

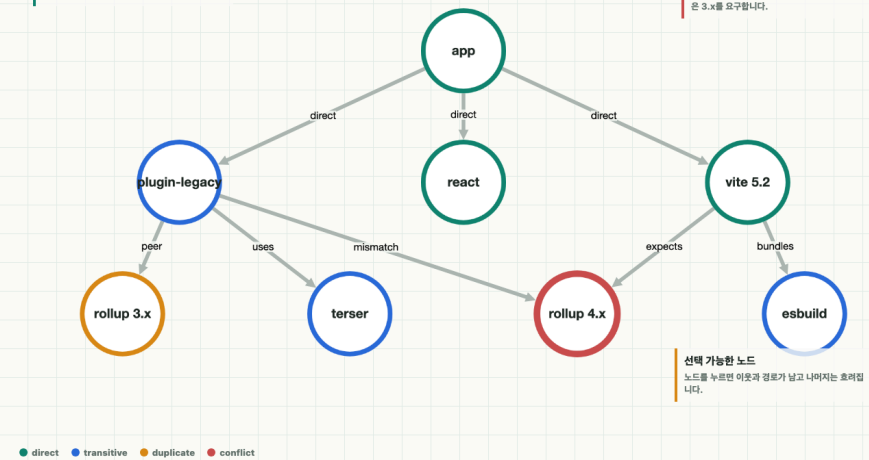
충돌 경로 강조

시작점은 app

직접 의존성과 transitive dependency를 한 그래프에서 파악합니다.

충돌은 Rollup peer range

Vite 5는 Rollup 4를 기대하고 legacy plugin은 3.x를 요구합니다.



cards 4 · box 0.05 · labels 0 · lib 1 · hand 0 · libscore 100 · state 1

What changed

복잡한 dependency network는 손으로 배치한 SVG보다 Cytoscape 같은 그래프 라이브러리가 layout, selection, neighbor/path highlight를 맡을 때 실제로 탐색 가능하다.

Cytoscape-backed dependency graph with real layout, selection, neighbor highlighting, and conflict-path toggle.

버튼이 문장만 바꾸는지, 시각 객체를 바꾸는지 확인

장에는 카드 목록보다 시간축에서 읽힌다.

비표, 지연 증가, 오류 폭증, 플렉, 회색을 같은 축에 놓으면 화면과 MTTR이 한눈에 보인다.

The chart displays three main metrics over time:

- latency** (orange line): Shows a significant increase during the degraded window, peaking around 09:14.
- requests** (blue line): Shows a steady decline during the degraded window, indicating a drop in traffic.
- errors** (red line): Shows a sharp increase during the degraded window, peaking around 09:14.

Key events and annotations:

- 09:00**: deploy (new payment API)
- 09:07**: 화면은 09:07부터 보일 때로부터 늦게 DB wait이 먼저 올라갑니다.
- 09:14**: 500s spike (alert fires)
- 09:14부터**: 사용자 방문은 09:14부터 오기들은 지연 증가 뒤에 떨어집니다.
- 09:07부터**: degraded window (DB waits rise, 09:07부터 rollback 요청이 보일 때까지)
- 09:21**: rollback starts
- 09:32**: stable again (latency and errors return)
- 복구는 한 정이 아니라 구간**: 플렉 뒤 지르기 안정화될 때까지 MTTR입니다.

리텐션 하락을 KPI가 아니라 cohort 표현에서 보인다

일리스 전후 cohort를 같은 색지에 놓으면 확실히 전체 문제가 아니라 특정 주치와 세그먼트 문제인지 분기됩니다.

cohort	일리스 경과가 관찰 기준 이 선 이후 cohort의 W1(W2를 이전 cohort로 비교)입니다.					
	W0	W1				
Feb 05	100	52	41	34	29	
Feb 12	100	51	40	33	28	
Feb 19	100	50	39	32	27	
Feb 26	100	43	30	24	20	
Mar 04	100	42	28	23	18	

**비록은 특정 설계가 시작
부터부터 나타나고 W0에서 주치가 더 떨어집니다.**

텍스트는 데이터 뒤에 붙임
오버랩을 포함한 같은 표본에서 다른 해석선이 드러납니다.

회색은 이전 baseline에 대한 선택 세그먼트

모바일: 하락폭이 더 큼

이전 baseline -8~9ppt

아키텍처 선택은 옵션 카드가 아니라 제약의 위치다

체크아웃 이벤트 처리는 자판 허용치와 버스트 크기를 넘어 후편이 어디까지 배워는지 드러냅니다.

제한된 작업

중간 작업

높은 작업

단일 API
작고 즉시 끝나는 작업

현재: API도 가능

serverless
간헐적 처리, 분할 가능

미크로: Queue를 이용

queue worker
버스트 흡수, 일정 분리

streaming pipeline
상시 대량 처리

200ms 응답 제한: 워크 단위씩 API 응답과 작업 완료율 분리해야 함

세분화 응답과 이벤트 처리를 분리해야 합니다.

보통 버스트 계속 이어옴 큰 피크

기울기는 점을 움직이는 화살표다

현재 점의 값이 높아지는 방향을 알려주면, gradient descent는 그 반대 방향으로 한 걸음 내딛습니다.

현재: 여기서는 오른쪽으로 올라감

기울기 방향
향수있어 올라가는 쪽입니다.

현재 x

다음 x

반대 방향으로 이동
 $x_{next} = x - lr \cdot \text{gradient}$

최소값 근처

목표는 낮은 곳
공식은 작은 값 이동을 알게 한 것입니다.

$lr = 0.18 \rightarrow$ **한 번 이동 완료** 작게 크게 **한 걸음 이동**

좋은 interaction은 설명 패널만 교체하지 않고, graph, timeline, decision surface, cohort grid 같은 primary object의 상태를 바꾼다.

CONCLUSION

현재까지의 결론과 다음 압력 지점

결론

좋은 설명은 예쁜 카드가 아니라 subject-native surface 위에서 관계, 상태, 근거, 변화가 직접 보이는 것이다.

현재 강점

알고리즘, API flow, code trace, science mechanism, timeline, architecture, analytics, library-backed network까지 anti-box 기준이 넓어졌다.

남은 압력

large hand-authored SVG debt, chart brushing, timeline zoom, maps, code editors, rich tables, 3D/physics, statistical uncertainty, narrative slide output을 추가해야 한다.